

Lecture - 01

Language processing System: (LPS)

A language processing system is a system software that translates programs written in one language into another language.

Source Program

↓
Pre-processor

↓
Compiler

↓
Assembler

↓
Linker

↓
Loader

↓
Memory

Pre-processor: — processes the source program before compilation. Handles `#include`, `#define` macros and comments.

Compiler: — Translates high level language into object code. Checks syntax and semantic errors. Produce object file.

10 - 10/10/2021

Assembler: — Convert assembly language into machine language.

Linker: — Links object files and library files together.
creates an executable file.

Loaders: — Load the executable program into main memory.

— Prepares the program for execution.

□ Difference between compiler and interpreter: —

Translation Method: —

• Compiler: Translate the entire program at once.

• Interpreter: Translate source code line by line.

Execution Speed: —

• Compiler: — Faster, execution, program runs after full compilation.

• Interpreter: — Slower, cause translating happens during execution.

Error handling:—

- Compiler: Shows error after compiling the whole code.
- Interpreter: Shows errors immediately line by line.

Compiler:—

Definition:— A compiler is a program that translates an entire source code into machine code before running it.

Common types of compiler:—

- I. Single pass
- II. Two-pass
- III. Multi-pass
- IV. Cross compiler.

Single Pass Compiler:—

A single pass compiler that reads, analyzes and translates the source program in one pass only.

Characteristics:—

- Read the source code only once
- Fast compilation.
- Limited error detection
- Little or no code optimization
- Requires that variables be declared before use

Example: Pascal (early version)

FORTAN (early version)

11. Two pass Compiler:—

A two-pass compiler that process the source program twice to generate machine code.

First-pass:—

- Analyzes the source program
- Builds symbol table
- Check syntax and semantic errors.

Second pass:—

- Uses the information from the first pass
- Generate object code
- Perform limited optimization.

Example: COBOL, Pascal (later version)

III. Multi-Pass Compiler:—

A multi-pass compiler is a compiler that scans and processes the source program more than two times to generate efficient machine code.

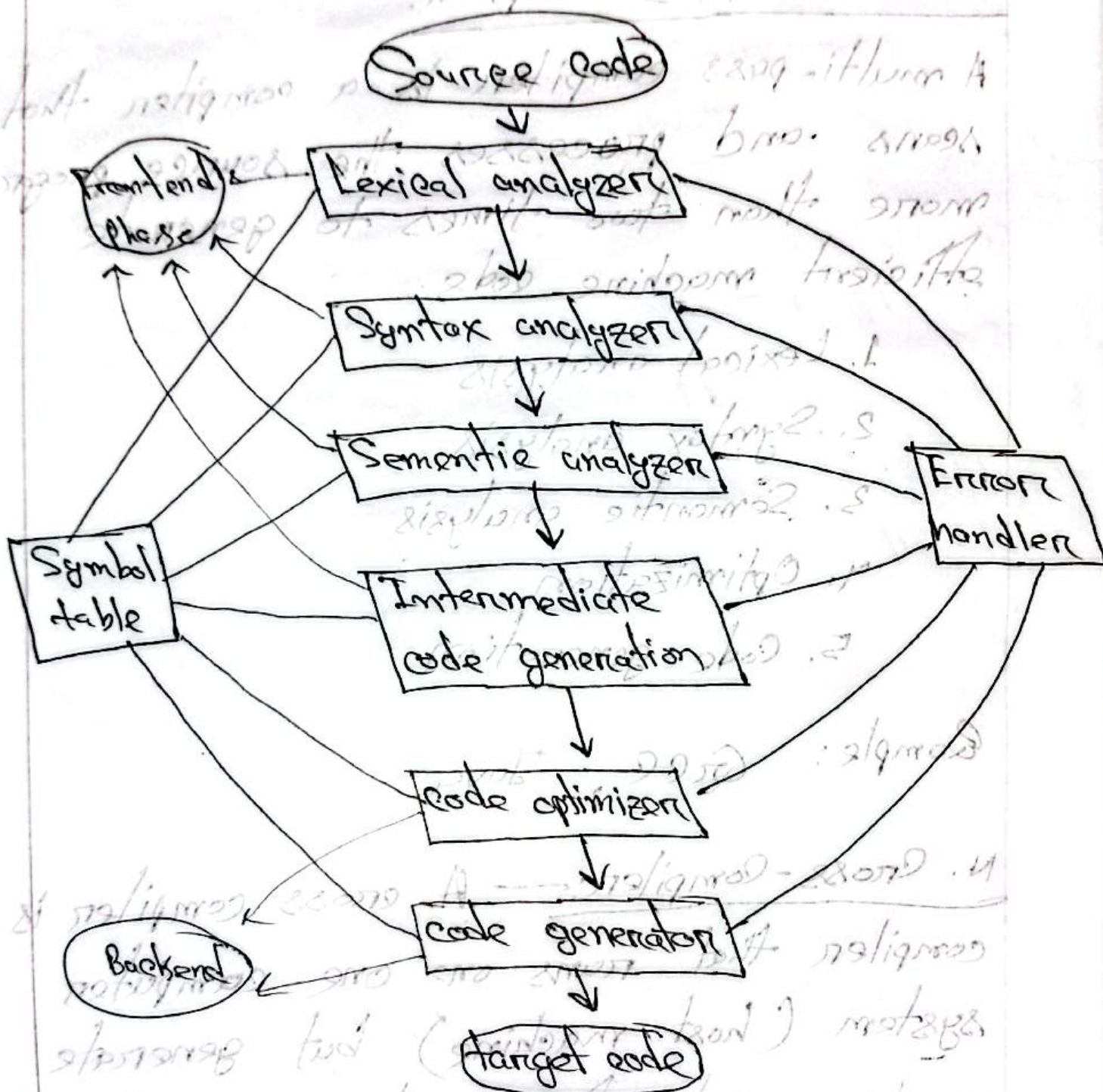
1. Lexical analysis
2. Syntax analysis
3. Semantic analysis
4. Optimization
5. Code generation

Example: C++, Java

N. Cross-Compiler:— A cross compiler is a compiler that runs on one computer system (host machine) but generate machine code for another computer system (target machine)

Lecture-02

Phases of a compiler :



1. Lexical Analysis (Scanner):

- Convert source program into sequence of tokens
 - Removes white spaces and comments
 - Groups characters into tokens (keywords, identifiers, operators, constants)

Example:

int a = 10 ;

Here,

int → keyword

a → identifier

= → operator

10 → constant

; → separator

2. Syntax analysis (Parser):

Checks whether the token sequence follows the grammar rules of the language.

- Builds parse tree or Syntax tree
- Detects syntax errors (missing semicolon, wrong structure).

Example: — `int a = ;`

Here,

syntax error.

3. Semantic Analysis:

- Checks the meaning of the program
 - Type checking
 - Variable declaration checking
 - Compatibility of operations.

Example: —

`int a;`

`a = "hello";`

Here,

Type mismatch error

4. Intermediate Code Generator: —

- Converts the source program into an intermediate representation (IR)
 - Machine independent
 - Makes optimization easier.

Example:—

(Three address code) $\rightarrow$ Max one operation and two operands

$$a = b + c + d \quad \times$$

$$a = b \quad \checkmark$$

$$x = y + z \quad \checkmark$$

5. Code Optimization:—

Improves the intermediate code to make it faster and more efficient.

— Removes unnecessary statements.

— Reduces execution time

— Reduce memory usage

Example:—

$$\begin{array}{l} x = 5 * 1 \\ \downarrow \\ x = 5 \end{array}$$

$$\begin{array}{l} x = 5 * 1 \\ \downarrow \\ x = R \\ \downarrow \\ R = 5 \end{array}$$

6. Code Generation :-

Converts optimized intermediate code into machine code.

- Selects registers
- Generate target machine instructions

output: executable program.

Symbol Table:- Stores information about variables, functions, data types, scope etc.

Error handler:- Detects, reports and recovers from errors during compilation.

Lecture-03
Date-17-02-26

Lexical Analysis

It converts the high level input program into a sequence of tokens.

— Lexical analysis can be implemented with the DFA.

— The output is a sequence of tokens that is sent to the parser for syntax analysis.

Token: A lexical token is a sequence of characters that can be treated as a unit in the grammar of the programming language.

— Type token (id, number, real...)

— Punctuation token (if, void, return...)

— Alphabetic tokens (keywords)

Example: —

float total, capital, rate;
total = capital + rate * 20;

Serial No.	Variable Name	Type
01	capital	float
02	total	float
03	rate	float

$id(total) = id(capital) + id(rate) * 20$

$\Rightarrow id_1 \neq id_2 + id_3 * 20$;

Grammar: —

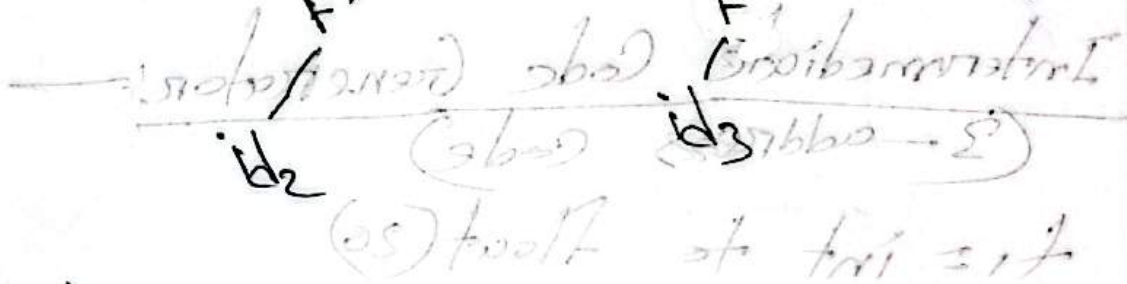
$S \rightarrow id = E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id \mid Num$

Arthropoda

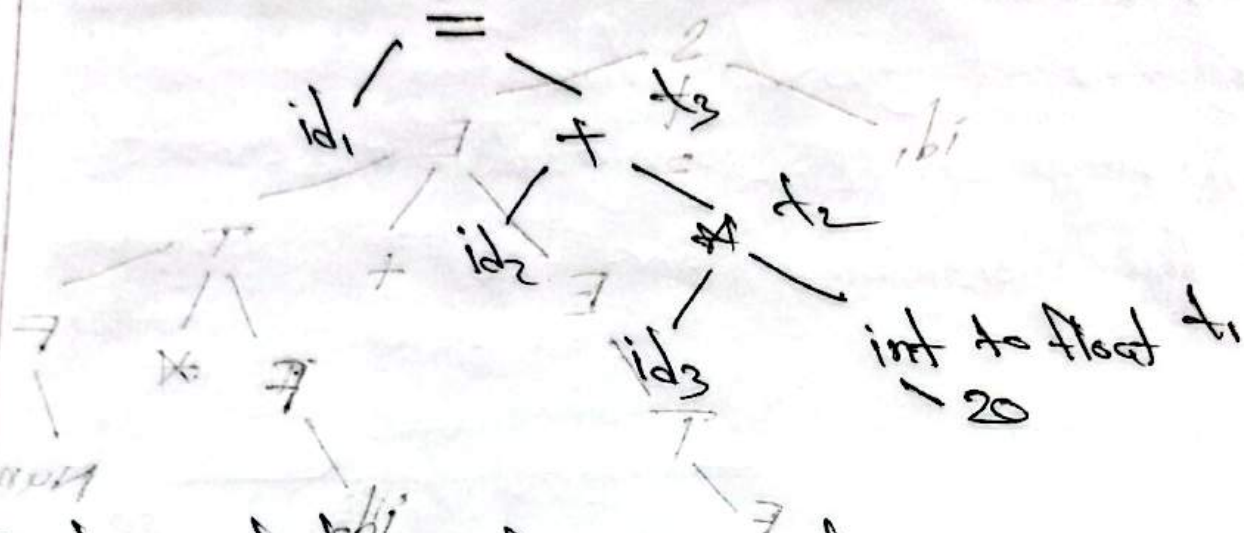

$$1\% \times 500 = 5\%$$

$$E = \frac{1}{2}$$

50. $\# \text{obi} = 1, \neq$

$$1 + 5i = 1i$$

Semantic Analyzer:



Intermediate Code Generator:

(3-address code)

$t_1 = \text{int to float}(20)$

$t_2 = id_3 * t_1$

$t_3 = id_2 + t_2$

$id_1 = t_3$

Code optimizer:

$t_1 = id_3 * 20.0$

$id_1 = id_2 + t_1$

limit of instruction A7D
 instruction

no - 2000
 25 - 5000

Code Generator:

A7D is for

LDA 1001 R1, #20.0
 MUL R1, R1, #20.0
 LDA R2, #10
 ADD R2, R1
 STR R2, 1001

states to be stored = 0

addresses to be stored = 3

initialization = 3

states to be stored = 0

states to be stored = 7

A7D is for

states to be stored = 3

states to be stored = 3

Lecture - 04
24-02-26

DFA $\rightarrow$ Deterministic Finite Automaton

What is DFA?

Ans: - A DFA is a theoretical machine used in automata theory to recognize patterns and languages. It is used in compiler design, lexical analysis and pattern matching.

5 tuples in DFA: -

Q = Finite set of states

Σ = Finite set of input symbols.

δ = Transition function

q_0 = initial state

F = set of final states.

Acceptability of a string by DFA: -

A string is accepted by a DFA if, after processing all input symbols starting from the initial state, the DFA reaches a final state. If it doesn't reach a final state, the string is rejected.

	$x=0$	$x=1$
$\rightarrow q_1$	q_3	q_2
q_2	q_4	q_1
q_3	q_1	q_4
q_4	q_2	q_3

$bbcc \rightarrow 0$
 $ccbb \rightarrow 1$

Truth Table

	0	1
0	0	1
1	1	0

q_3
 initial state

Transitions:

$\delta(q_1, 110101)$

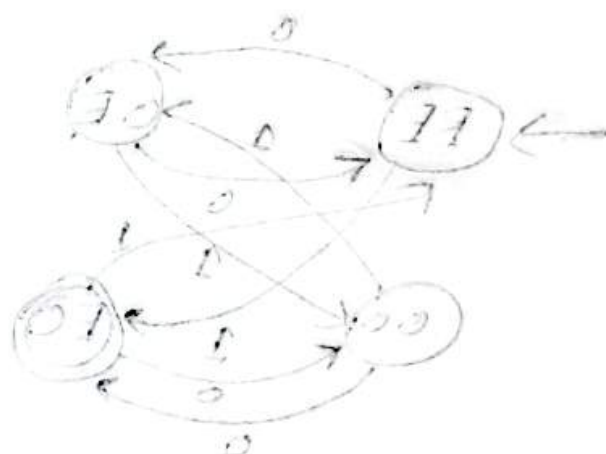
$\delta(q_2, 10101)$

$\delta(q_1, 0101)$

$\delta(q_3, 101)$

$\delta(q_4, 01)$

$\delta(q_2, 1) = q_1$



q_1 is final state

Question:

Design one DFA which takes 0's and 1's as input string and accepts that string which have even number of 0's and odd number of 1's.

Solution:-

0 → odd

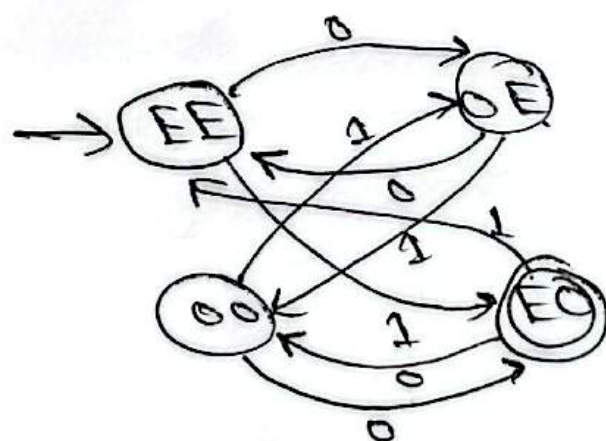
E → even

There will be 4 case

0	1
E	E

0	E
E	O
0	0

→ accepted state



State limit is 1P

1P = (1, 1P) 2

1P) 2

→ 2P

(101011, 1P) 2

(10101, 1P) 2

(1010, 1P) 2

(101, 1P) 2

(10, 1P) 2

so if we do not know DFA we might not accept the string which have 1P to number the 1P to number the 1P

Lecture-05

31-03-26

NFA to DFA conversion:

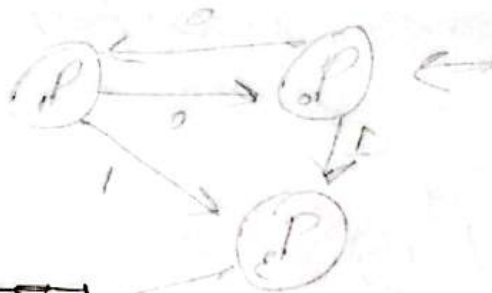
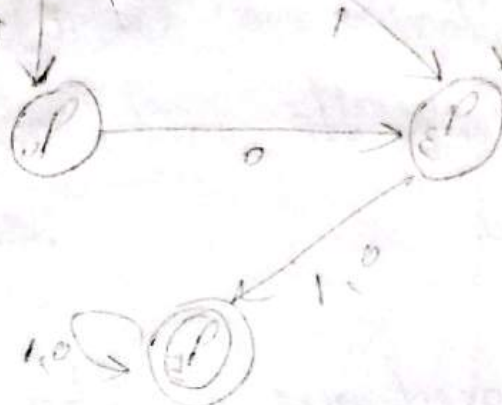
$$M = (\{q_1, q_2, q_3, q_4\}, \{0, 1\}, \delta, q_1, \{q_4\})$$

	0	1
$\rightarrow q_1$	$q_1 q_2$	q_1
q_2	q_3	q_2
q_3	q_4	q_4
(q_4)		q_3

NFA

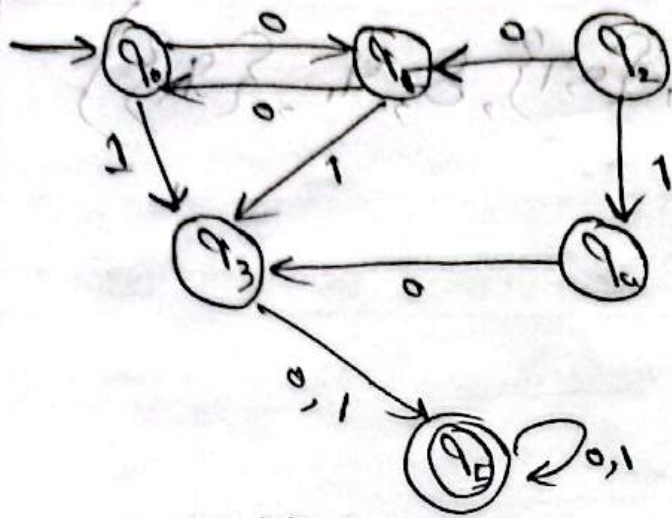
	0	1
q_1	$[q_1 q_2]$	$[q_1]$
$[q_1 q_2]$	$[q_1 q_2 q_3]$	$[q_1 q_2]$
$[q_1 q_2 q_3]$	$[q_1 q_2 q_3 q_4]$	$[q_1 q_2 q_4]$
$[q_1 q_2 q_3 q_4]$	$[q_1 q_2 q_3 q_4]$	$[q_1 q_2 q_3 q_4]$
$[q_1 q_2 q_4]$	$[q_1 q_2 q_3]$	$[q_1 q_2 q_3]$

DFA Transition table.



20-07-2011
25-03-12

Minimization of Finite Automata

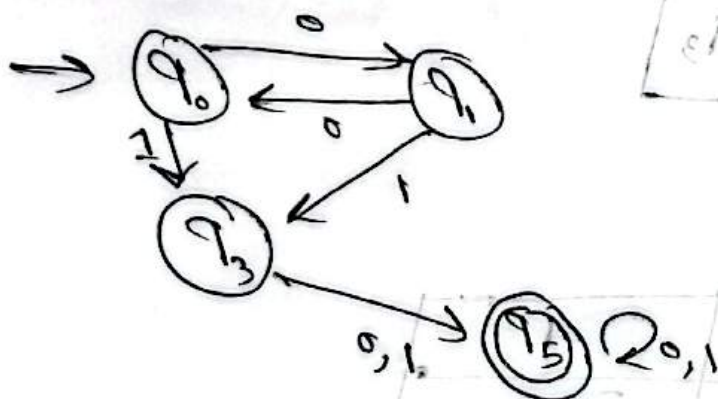


DFA

$\{q_0, q_1, q_3, q_4\} = M$

	1	0	
q_0	q_1	q_0	q_0
q_1	q_3	q_0	q_3
q_3	q_5	q_5	q_5
q_4	q_3	q_5	q_5
q_5	q_5	q_5	q_5

A74



mDFA

Definition:

Two states q_1 and q_2 are equivalent (denoted $q_1 \equiv q_2$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both are non-final states for all x .

$$x \in \Sigma^*$$

* Two states q_1 and q_2 are k -equivalent ($k \geq 0$) if both $\delta(q_1, x)$ and $\delta(q_2, x)$ are final states or both non-final states for all strings x of length k or less.

Property 1: — The relations we have defined i.e. equivalence and k -equivalence are equivalence relation i.e. they are reflexive, symmetric and transitive.

Property 2: Partitions are denoted by Π or Π_k element of Π_k are k -equivalence classes.

Property 3: — If q_1 and q_2 are k -equivalence for all $k \geq 0$, then they are equivalent.

Property 4: — If q_1 and q_2 are $(k+1)$ -equivalent, then they are k -equivalent if $\Pi_n = \Pi_{n+1}$ for some n .

* $\Delta \in \Sigma$

	0	1
0	q_1	q_2
1	q_3	q_4
2	q_5	q_6
3	q_7	q_8
4	q_9	q_{10}
5	q_{11}	q_{12}
6	q_{13}	q_{14}
7	q_{15}	q_{16}
8	q_{17}	q_{18}

$$\Pi_0 = \{ \{q_3\}, \{q_1, q_2, q_4, q_5, q_6, q_7, q_8\} \}$$

$$\Pi_1 = \{ \{q_3\}, \{q_4, q_5\}, \{q_2, q_8\}, \{q_1, q_5, q_7\} \}$$

$$\Pi_2 = \{ \{q_3\}, \{q_4, q_5\}, \{q_2, q_8\}, \{q_1, q_5\}, \{q_7\} \}$$

$$\Pi_3 = \{ \{q_3\}, \{q_4, q_5\}, \{q_2, q_8\}, \{q_1, q_5\}, \{q_7\} \}$$

$$\Pi_2 = \Pi_3 \rightarrow 2\text{-equivalent}$$

Since $\Pi_1 \neq \Pi_0$

	0	1
$[9, 95]$	$[9, 93]$	$[9, 96]$
$[9, 95]$	$[9, 7]$	$[9, 3]$
$[9, 96]$	$[9, 3]$	$[9, 7]$
$[9, 3]$	$[9, 7]$	$[9, 95]$
$[9, 3]$	$[9, 95]$	$[9, 3]$

x

Lecture - 06

Date: 15-04-26

Context Free Grammar (CFG)

A context free Grammar is a formal grammar which is used to generate all the possible patterns of strings in a given formal language.

It is defined as four tuples:—

$$G = (V, T, P, S)$$

$G \Rightarrow$ is a grammar, which consists of a set of productions rules, to generate the strings of a language.

$T \rightarrow$ is a final set of terminal symbols.

$V \rightarrow$ is a final set of non-terminal symbol

$P \rightarrow$ is a set of production rules

$S \rightarrow$ is the start symbol.

Question:—

Identify the terminals, non-terminals, start variable from the following

I. $E \rightarrow E + T \mid T$ II. $S \rightarrow (L) \mid a$

$T \rightarrow T * F \mid F$

$L \rightarrow L \mid S$

$F \rightarrow (E) \mid id$

Question:—

Given language

I. $L = \{ a^n \mid n \geq 0 \}$

II. $L = \{ a^n b^n \mid n \geq 1 \}$

III. $L = \{ a^n \mid n \geq 1 \}$

IV. Set of all string which length atleast

V. Starts with 'a' and end with 'b'.

Solution:—

1. $L = \{a^n \mid n \geq 0\}$

output = $\{\epsilon, a, aa, aaa, \dots\}$

production: $A = aA \mid \epsilon$

2. $L = \{a^n b^n \mid n \geq 1\}$

output = $\{ab, aabb, aaabbb, \dots\}$

production: $A = ab \mid aAb$

4. $a^* \rightarrow \epsilon, a, aa, aaa, \dots$ with ϵ

$a^+ \rightarrow a, aa, aaa, \dots$ without ϵ

$S \rightarrow AAB$

$A \rightarrow a \mid b$

$B \rightarrow \epsilon \mid aB \mid bB$

5. $ab, aab, abab$ How short? $\sqrt{}$

$$S \rightarrow aAb$$

$$A \rightarrow \epsilon \mid aA \mid bA \quad \{0 \leq m/n\} = 1$$

Lecture - 07

Date: 19-04-26

Left most and Right most derivation in CFG:-

Left most Derivation:-

A derivation $A^* \Rightarrow w$ is called a left most derivation if we apply a production only to the left most variable at every step.

Right most Derivation:-

A derivation $A^* \Rightarrow w$ is called a right most derivation if we apply a production only to the right most variable at every step.

Question:

Consider the following Grammar G

$$S \rightarrow aAS$$

$$S \rightarrow a$$

$$A \rightarrow SbA$$

$$A \rightarrow SS$$

$$A \rightarrow ba$$

For the following input strings: $aabbba$

(a) Leftmost derivation

(b) Rightmost derivation

(c) Derivation tree.

Solution:

(a) $S \rightarrow aAS$

$$aSbAS$$

$$aabbAS$$

$$aabbbaS$$

$$aabbbaa$$

Left most

$$\textcircled{b} S \rightarrow aAS$$

Consider the following

$$aAa$$

$$aSbAa$$

$$aSbbaa$$

$$aabbbaa$$

$$2A \rightarrow 2$$

$$p \rightarrow 2$$

$$Ad2 \leftarrow A$$

$$22 \leftarrow A$$

$$pd \leftarrow A$$

Right most

Question:

Consider the context free Grammar

$$S \rightarrow SS+ / SS* / a$$

1. Show how the string $aataa$ can be generated by these grammar in left and right most derivation in 2

CFG

$$2A \rightarrow 2 \quad \textcircled{a}$$

ii. Construct a parse tree for this string.

$$2Ad \rightarrow 2$$

$$2add \rightarrow 2$$

$$add \rightarrow a$$

from the

Solution:

① L.M

$S \rightarrow SS^*$

$$SS + S^*$$
$$a_s + s^*$$

aa + S +

$$aa + a*$$

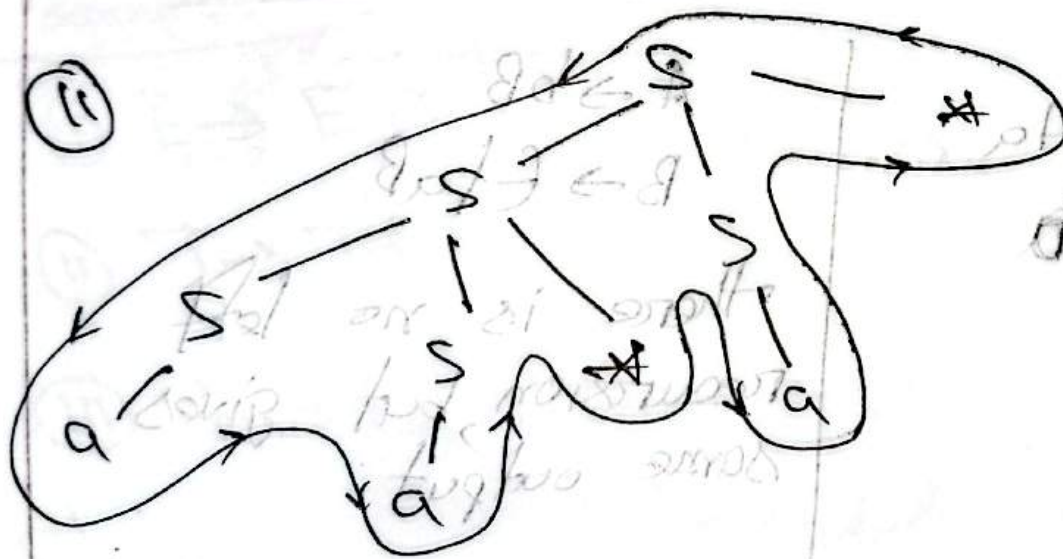
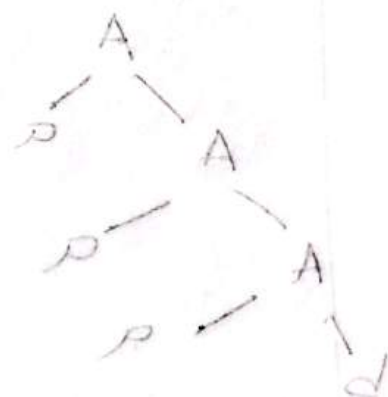
$SS \rightarrow S$

202

14.55 + 9.4

Safat

aa+ax


$$D \leftarrow A$$
$$d \leftarrow A$$


Feb 29th 1900

mi 1861370927

Example: — $\frac{1}{2} \log x$ $\frac{1}{2} \log y$

$$d/PA \leftarrow 2$$

3/62 ← A

Lecture - 8

Date: 26-04-26

☐ Predictive Parsing Table: 1/10/2

- Calculate first and follow set M1 ①

- There shouldn't any

Left recursion

Left factor

☐ Left Recursion: *22 ← 2
*2 + 22 ← 2
*2 + 20
*2 + 00
*0 + 00

Example: -

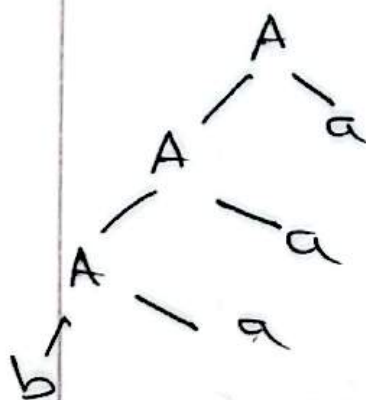
$A \rightarrow Aa$

$A \rightarrow a$

$A \rightarrow bB$

$B \rightarrow \epsilon / aB$

there is no left recursion, but gives same output ①



there indirect left recursion in

following example: -

$S \rightarrow Aa / b$

$A \rightarrow Sd / \epsilon$

$S \rightarrow Sda / b$

* If $A \rightarrow A\alpha \mid \beta$ — not left recursive

then,

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \epsilon \mid \alpha A' \end{aligned}$$

equation for remove left recursion

$A' \rightarrow$ is a new production

Example: —

① $E \rightarrow E+T \mid T$

② $T \rightarrow T * F \mid F$

③ $F \rightarrow (E) \mid id$

Solution: —

① $E \rightarrow TE'$

$E' \rightarrow \epsilon \mid +TE'$

② $T \rightarrow FT'$

$T' \rightarrow \epsilon \mid *FT'$

③ There is no left recursion

□ Left factor: —

$S \rightarrow i c t s \mid i c t s e s$

$c \rightarrow b$

$S \rightarrow i c t s s'$

$S \rightarrow \epsilon \mid e s$

$A a \leftarrow A$

$'A' / 3 \leftarrow 'A'$

work on $a i \leftarrow 'A'$
notation

□ Calculation of first Set: —

1. If x is terminal, then $FIRST(x)$ is $\{x\}$ (i)

2. If $x \rightarrow \epsilon$ is a production, then add ϵ to $FIRST(x)$ (ii)

3. If x is non-terminal and $x \rightarrow y_1 y_2 y_3 \dots y_k$ is a production, then place a in $FIRST(x)$, if, for some i , a is in $FIRST(y_i)$ and ϵ is in all of $FIRST(y_1) \dots FIRST(y_{i-1})$ that is $y_1 \dots y_{i-1} \Rightarrow \epsilon$, if ϵ is in (iii)

$\text{FIRST}(Y_j)$ for all $j=1,2,3$ symbols
then add ϵ to $\text{FIRST}(X)$

□ Calculation of follow:

1. Place $\$$ in $\text{FOLLOW}(S)$, where S is the start symbol and $\$$ is the input right end marker.
2. If there is a production, $A \rightarrow \alpha B \beta$, then everything in $\text{FIRST}(\beta)$ except for ϵ is placed in $\text{FOLLOW}(B)$.
3. If there is a production $A \rightarrow \alpha \beta$ or a production $A \rightarrow \alpha B \beta$ where $\text{FIRST}(\beta)$ contains ϵ , then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

Example:

$$I. E \rightarrow E + T \mid T$$

$$II. T \rightarrow T * F \mid F$$

$$III. F \rightarrow (E) \mid id$$

calculate FIRST and FOLLOW set.

Solution:

Removing left recursion and
left factor:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \epsilon$$

$$F \rightarrow (E) \mid id$$

	FIRST	FOLLOW
$E \rightarrow TE'$	$\{id, (\}$	$\{ \$,) \}$
$E' \rightarrow +TE' \mid \epsilon$	$\{ +, \epsilon \}$	$\{ \$,) \}$
$T \rightarrow ET'$	$\{id, (\}$	$\{ +, \$,) \}$
$T' \rightarrow *FT' \mid \epsilon$	$\{ *, \epsilon \}$	$\{ +, \$,) \}$
$F \rightarrow (E) \mid id$	$\{id, (\}$	$\{ *, +, \$,) \}$

Lecture - 09

Date: 30-04-26

Construction of Predictive Parsing Table:

Input: Grammar G

Output: Parsing table M

Method: —

1. For each production $A \rightarrow \alpha$ of the grammar, do steps 2 and 3.
2. For each terminal a in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$

W01107

T2917

3. if ϵ is in $\text{FIRST}(a)$, add $A \rightarrow a$ to $M[A, b]$ for each terminal b in $\text{FOLLOW}(A)$. If ϵ is in $\text{FIRST}(a)$ and $\$$ in $\text{FOLLOW}(A)$, add $A \rightarrow a$ to $M[A, \$]$

4. Make each undefined entry of M be error.

Question: —

20-01-2021
Date: 20-01-21

$E \rightarrow TE'$

$E' \rightarrow +TE' / \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' / \epsilon$

$F \rightarrow (E) / id$

construct a predictive parsing table.

Example: $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$

(b) M of A is

Solution:

calculate first and follow set :-

Grammmere	First	Follow
$E \rightarrow TE'$	$\{id, (\}$	$\{ \$,) \}$
$E' \rightarrow +TE' / \epsilon$	$\{ +, \epsilon \}$	$\{ \$,) \}$
$T \rightarrow FT'$	$\{ id, (\}$	$\{ +, \$,) \}$
$T' \rightarrow *FT' / \epsilon$	$\{ *, \epsilon \}$	$\{ +, \$,) \}$
$F \rightarrow id / (E)$	$\{ id, (\}$	$\{ *, +, \$,) \}$

Predictive Parsing table:

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Predictive Parser:

Set ip to point to the first symbol
of $w\$$
repeat

begin

Let x be the symbol and/or the
symbol pointed to by ip ;

if x is a terminal or $\$$ then

if $x = a$ then
pop x from the stack
advance ip

else
ERROR();

else x is a non-terminal

if $M[x, a] = X \rightarrow \gamma_1 \gamma_2 \gamma_3 \dots \gamma_n$ then
begin

pop x from the stack
push $\gamma_n \gamma_{n-1} \gamma_{n-2} \dots \gamma_1$ onto
the stack with γ_1

Output on the top
the production

until
 else
 ERROR();
 until x = \$

— x —

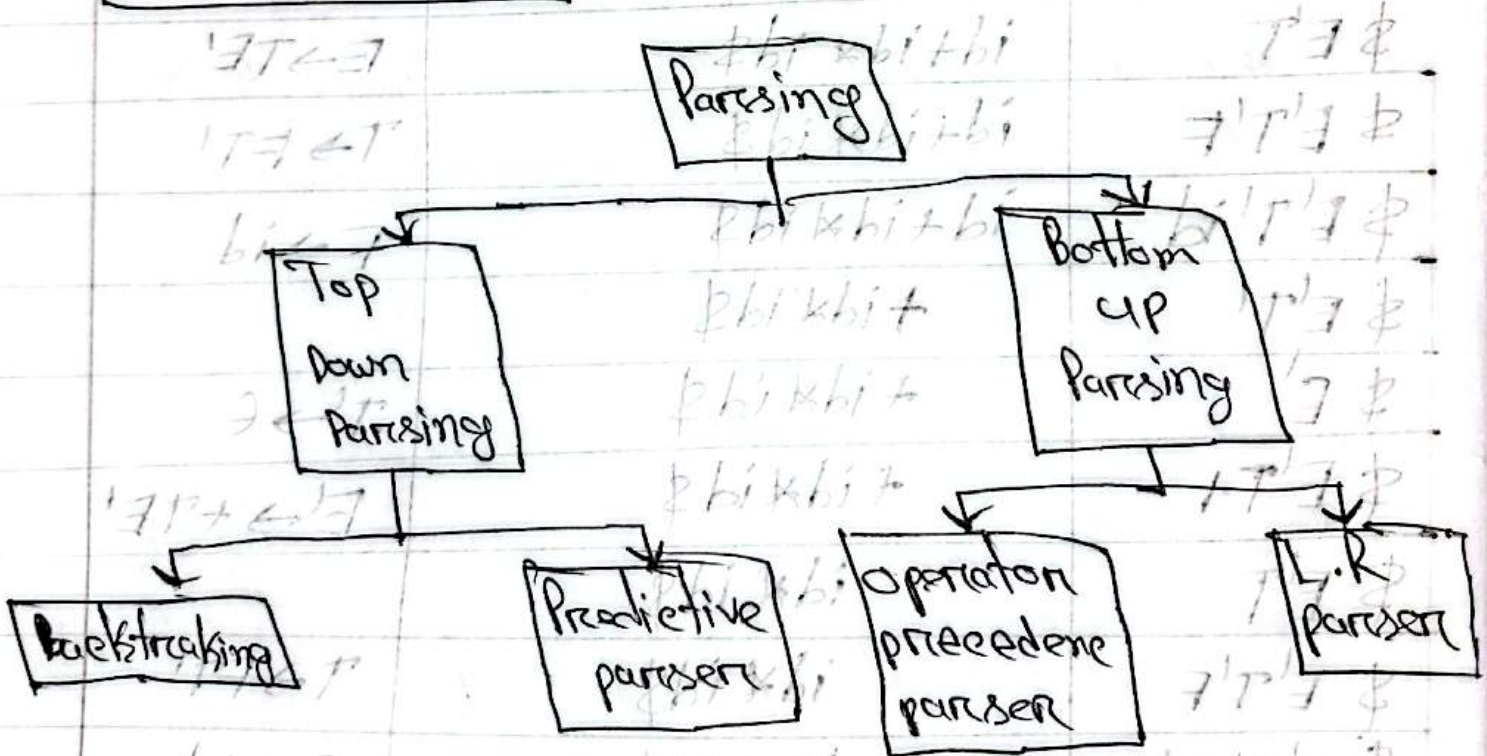
STACK	INPUT	OUTPUT
\$E	id+id*id\$	
\$E'	id+id*id\$	E → TE'
\$E'T'F	id+id*id\$	T → FT'
\$E'T'id	id+id*id\$	F → id
\$E'T'	+id*id\$	
\$E'	+id*id\$	T' → E
\$E'T+	+id*id\$	E' → +TE'
\$E'T	id*id\$	
\$E'T'F	id*id\$	T → FT'
\$E'T'id	id*id\$	F → id
\$E'T'	*id\$	
\$E'T'F*	*id\$	T' → *FT'
\$E'T'F	id\$	

STACK	INPUT	OUTPUT
\$ E' id	id \$	$E \rightarrow id$
\$ E'	\$	
\$ E'	\$	$E' \rightarrow E$
\$	\$	$E' \rightarrow E$

$\$ = \times / \text{literal}$

Lecture - 10

Date: 04-05-26



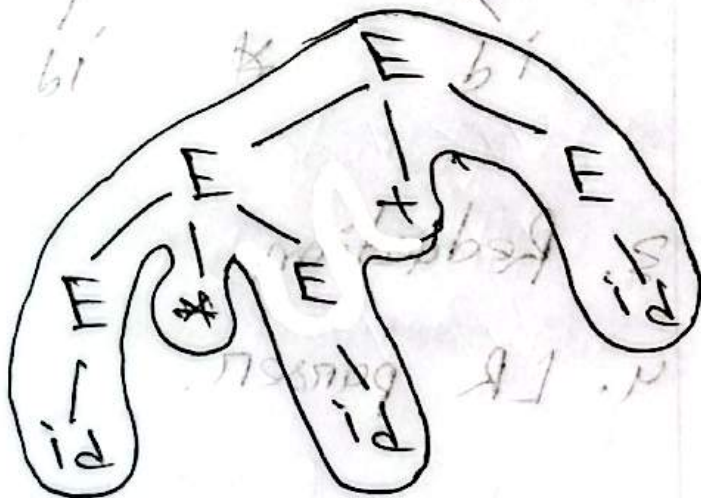
□ Top Down: —

1. Parser starts construction the parse tree from start symbol and tried to transform the start symbol of the input symbol.

$$2. E \rightarrow E + E \mid E * E \mid id$$

$$w \rightarrow id * id + id + id$$

$$\begin{aligned} \therefore E &\rightarrow E + E + id \\ &\rightarrow E * E + E \\ &\rightarrow id * E + E \\ &\rightarrow id * id + E \\ &\rightarrow id * id + id \end{aligned}$$



3. Expansion

4. LL parse.

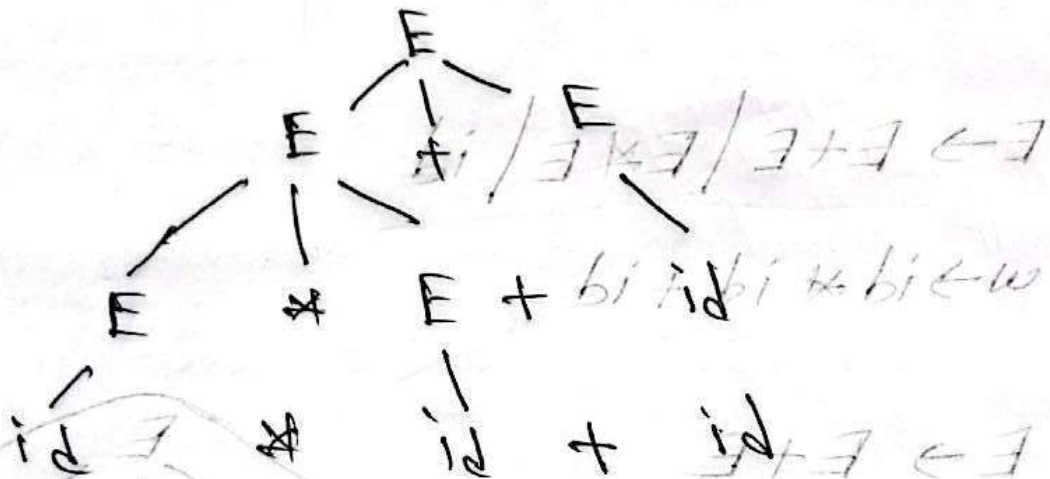
$$E \rightarrow E + E \mid E * E \mid id$$

$id * id + id$: printer

□ Button Up: —

1. Start with input symbol and tries to construct parse tree upto start symbol.

2.



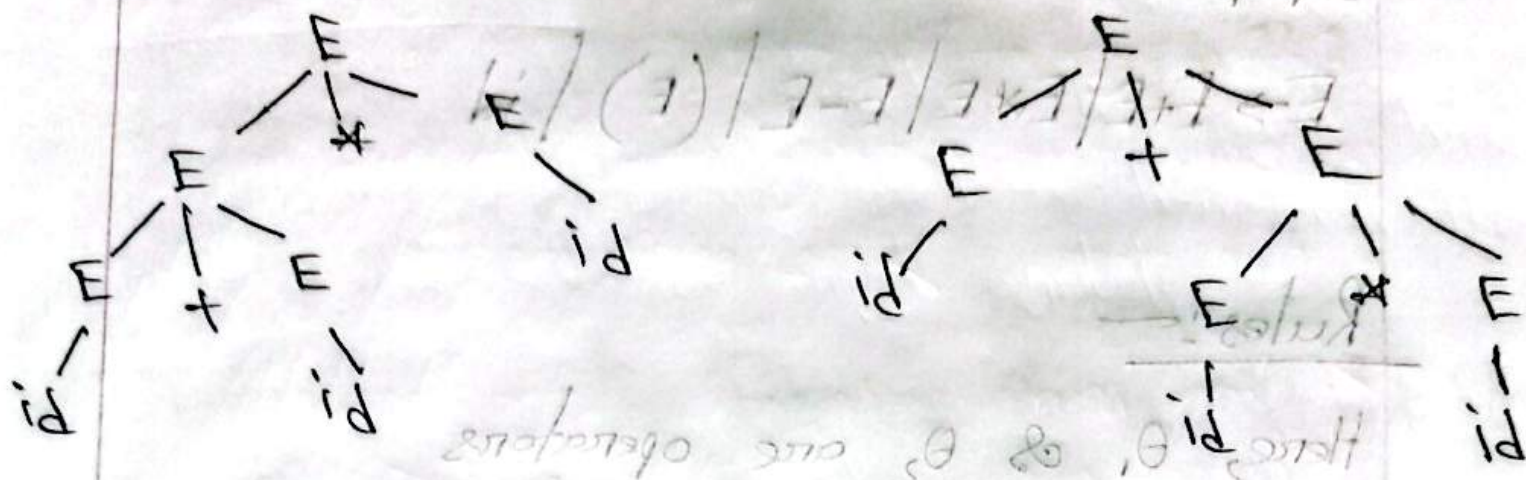
3. Reduction

4. LR parser.

Example: —

$$E \rightarrow E * E \mid E + E \mid id$$

String: id * id + id



Here arise ambiguous problem. It can be solve using operator precedence parsing.

Operator Precedence Parsing (O.P.P):

A grammar G is operator Grammar if it has the following properties:-

- Production shouldn't contain ϵ on it's right side
- There shouldn't be two adjacent non-terminal at the right side of production.

Example:-

$$E \rightarrow EAE \mid (E) \mid id$$

$$A \rightarrow + \mid - \mid *$$

O.P.P

$$E \rightarrow E + E \mid E * E \mid E - E \mid (E) \mid id$$

Rules:-

Here, θ_1 & θ_2 are operators

* if precedence of θ_1 is higher than θ_2
then $\theta_1 > \theta_2$, $\theta_2 < \theta_1$

* if precedence of θ_1 and θ_2 are same
then $\theta_1 > \theta_2$ and $\theta_2 > \theta_1$ if left
associative A

$\theta_1 < \theta_2$ and $\theta_2 < \theta_1$ if Right
Associative

* Other rules are

$$\begin{aligned} ' & < id < \$ \\ \$ & < ' \\ \$ & < id < ' \end{aligned}$$

Lecture-11

Date: 05-05-26

* Consider the following grammar and construct the operator precedence parse: —

$$E \rightarrow EAE \mid id$$

$$A \rightarrow + \mid *$$

Then parse the following string: $id + id * id$

Step 1: — convert the given grammar into operator precedence grammar

$$E \rightarrow E + E \mid E * E \mid id$$

Step 2: — Both end of the string add \$ symbol then construct operator precedence table

$$\$ id + id * id \$$$

terminals symbols are: $\{ \$, id, +, *, \$ \}$

	id	+	*	\$
id		>	>	>
+	<	>	<	>
*	<	>	>	>
\$	<	<	<	

Step 3:- Parse the given string using following rules:-

1. Scan the string from left end until the first $>$ is encountered $\leftarrow A$

2. Then scan backwards (to the left) over any $=$ until a $<$ is encountered.

3. Then handle everything to the left of the first $>$ and to the right of the $<$ encountered in step (2), including any inverting or surrounding non-terminals, \$ on \$ means parsing is successful

\$ <id> + <id> * <id> \$

\$ ^ + ^ id> * <id> \$

\$ ^ + * ^ <id> \$

\$ ^ + ^ ^ \$

\$ ^ + ^ \$

successful

Lecture - 12

Date: 10-05-26

Stack Implementation of Shift-Reduce Parsing: —

A convenient way to implement a shift reduce parser is to use a stack to hold grammar symbols and an input buffer to hold the string w to be parsed. We use $\$$ to mark the bottom of the stack and also the right end of the input. Initially, the stack is empty, the string w is on the input, as follows,

<u>Stack</u>	<u>input</u>
$\$$	$w\$$

Successful completion of parsing

<u>Stack</u>	<u>input</u>
$\$ s$	$\$$

Primary Operation: —

1. Shift
2. Reduce
3. accept
4. ERROR

1. In a shift action, the next input symbol is shifted onto the top of stack.
2. In a reduce action, the parser knows the right end of the handle is at the top of the stack. It must locate the left end of the handle within the stack and decide with what non-terminal to replace the handle
3. Accept
4. Error.

Grammar

$$E \rightarrow E + E \mid E * E \mid id$$

String: $id_1 + id_2 * id_3$

Stack	Input	Action
1. \$	$id_1 + id_2 * id_3 \$$	Shift
2. \$ id_1	$+ id_2 * id_3 \$$	Reduced by $E \rightarrow id$
3. \$ E	$+ id_2 * id_3 \$$	Shift
4. \$ $E +$	$id_2 * id_3 \$$	Shift
5. \$ $E + id_2$	$* id_3 \$$	Reduced by $E \rightarrow id$
6. \$ $E + E$	$* id_3 \$$	Shift
7. \$ $E + E *$	$id_3 \$$	Shift
8. \$ $E + E * id_3$	\$	Reduced by $E \rightarrow id$
9. \$ $E + E * E$	\$	Reduced by $E \rightarrow E * E$
10. \$ $E + E$	\$	Reduced by $E \rightarrow E + E$